Part 2 — Do CC

Programmable Congestion Control with DOCA PCC

Programming SmartNICs: From Packet Processing to Programmable Transport

ACM SIGCOMM 2026

Fernando Ramos Muhammad Shahbaz Mina Tahmasbi Arashloo Mario Baldi
Francisco Pereira Bilal Saleem Tyler Liu Chenxingyu Zhao

INESC-ID, Instituto Superior Técnico, University of Lisbon University of Michigan University of Waterloo
NVIDIA Purdue University University of Washington

In **Part 1** (`doca-flow.pdf`) you programmed the NIC to *mark* congestion — you set the **CE** bit on packets in the data plane. In this part you write the other side of that story: the **congestion-control algorithm that reacts to those marks**, deciding how fast a flow is allowed to send — and you run it on a set of tiny processors *inside* the NIC called the **DPA**.

By the end you will have **written the two reactions at the heart of a reaction-point controller**, watched a flow's send rate **collapse** the moment congestion appears and **recover** when it clears, and tuned how aggressively it reacts.

> **Prerequisites.** - You have done **Part 1** (`doca-flow.pdf`) — PCC reacts to the ECN marks you produced there. - You are on the **Arm cores** of a BlueField-3 with this repo checked out and sudo access. - One firmware knob, **`USER_PROGRAMMABLE_CC=1`**, must be live for any PCC program to start — on the tutorial cards this is already set for you (details in Appendix A).

Step 1 — Where the algorithm runs

A DOCA PCC program comes in two pieces that run in two different places:

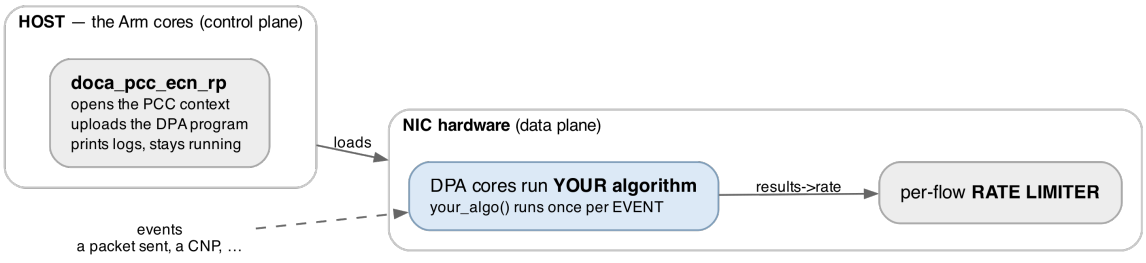

Figure 1: The two halves of a DOCA PCC program. The host side only loads and supervises; every packet-time decision happens on the DPA, inside the algorithm you write.

- **The host program** (`host/pcc_ecn_rp.c`) is just a **loader and supervisor**. It uploads your compiled algorithm to the NIC, opens the PCC context, and then sits there keeping it alive and printing logs. It does **not** run the algorithm itself.
- **The algorithm** runs on the **DPA** — the *Data-Path Accelerator*, a cluster of small, highly parallel processors inside the BlueField-3. That is where the C code you'll edit actually executes.

  INFO — what is the DPA, and why not the Arm cores? The Arm cores are general-purpose Linux CPUs; they are too far from the wire to make a per-flow rate decision fast

enough. The DPA sits right on the data path and is built for exactly this kind of tiny, frequent, reactive computation. You write the algorithm in C, a special compiler (`dpacc`) turns it into a DPA program, and the host loader ships it onto the NIC. (More in Appendix A.)

What your algorithm actually outputs: a rate. Its whole job is to set **one number per flow**: a target send **rate**. It never touches a packet and never paces anything itself. It writes the rate into a `results` struct, and when it returns, the NIC programs that flow's **hardware rate limiter** to that value. From then on the *hardware* paces the packets — at line speed, with your algorithm nowhere in the loop — until the next event, when your code runs again to revise the number.

> **INFO — rate is a fraction, not a bits-per-second.** The rate is a fixed-point number where **`1 << 20` (= 1,048,576) means "100% of line rate."** So `524288` is 50%, and a small floor value is a near-stop. You will see constants like this in the code; just read `1 << 20` as "full speed."

This split is the key idea of PCC: **you write the *policy* (what the rate should be), the NIC hardware does the *work* (pacing every packet).**

Step 2 — Build it and run it once

Before opening the algorithm, **build the whole thing and run it once** — to prove your toolchain works and to *see* the two halves in action: the host loader compiling your DPA program, uploading it to the NIC, and running it live.

> **NOTE — the algorithm is deliberately incomplete right now.** The controller you're about to run is a **scaffold**: the two reactions that move the rate are left blank (you write them in Step 4). So it loads, runs, and *sees* congestion — but it won't change the rate yet. That's expected; this step is about the plumbing, not the policy.

Try it yourself! Build the controller and watch it run.

Build. On the card's Arm cores, from the repo root:

```
cd doca-2 && meson setup build && ninja -C build
# => build/doca-pcc-ecn/doca_pcc_ecn_rp   (the host loader; the DPA program is compiled into
↪  it)
```

That one step compiles the C you'll edit in Step 4 into a **DPA image** and links it into the host loader. It builds fine even with the two reactions still blank.

Load the controller on PF1, in the **foreground**, so all its output prints right here:

```
sudo stdbuf -oL ./build/doca-pcc-ecn/doca_pcc_ecn_rp -d mlx5_1 -l 50
```

> **INFO — the flags.** `-d mlx5_1` picks PF1 (the sender's uplink — the "reaction point"); `-l 50` is a chatty log level; `stdbuf -oL` keeps the output line-buffered so it appears as it happens. It runs in the **foreground** — you watch it live and stop it with **Ctrl-C** (SIGINT, the graceful stop; never `kill -9` it — see Debugging tips).

It opens the PCC context, uploads your DPA program, and then waits for events. With no traffic yet it just sits there — a controller with nothing to react to has nothing to do.

Give it something to see. In **other terminals** (leave the controller running), start the Part 1 marker so packets are CE-marked and the receiver sends CNPs, then drive traffic with `benchmark.sh` — the loopback is already up from Part 1:

```
# your finished Part 1 marker: CE-mark every packet
sudo ./build/doca-flow/doca_flow_ecn -- --percent 100
# from the repo root — server + client together, with a live throughput chart
```

```
./scripts/benchmark.sh
```

What you should see. Back in the controller's terminal, its `PURE_ECN` trace scrolls by — proof your algorithm is running on the DPA and seeing the CNPs:

```
PURE_ECN cnp=1    rate=1048576
PURE_ECN cnp=501  rate=1048576
PURE_ECN cnp=1001 rate=1048576
# it sees the CNPs, but the rate never moves — no reactions yet
```

Those lines are mixed in with the loader's chatter; if it's too busy to read, stop it and reload with `| grep --line-buffered PURE_ECN` appended. That flat rate is exactly the point of the checkpoint: **your code compiled, loaded onto the NIC's DPA, and is running** — it just doesn't *react* yet. Making it react is Step 4. (`1048576` is `1 << 20` — full line rate, from Step 1.)

Stop it. In the controller's terminal press **Ctrl-C**, then stop the marker and traffic in the other terminals.

Step 3 — How the controller reacts

You ran this program in Step 2; here is what it does and where *your* code plugs in. The whole thing is a textbook **DCQCN** loop — the classic additive-increase / multiplicative-decrease pattern — and you write its two reactions.

The host half: `main()` in `host/pcc_ecn_rp.c`. You won't edit it, but its flow shows exactly where the DPA takes over:

1. **Read the two flags** — `-d <device>` (which NIC, e.g. `mlx5_1`) and `-l <level>` (log verbosity).
2. **Open that device** — `open_pcc_device()` finds the IB device that *supports PCC* and opens it.
3. **Create a PCC context** — `doca_pcc_create()`.
4. **Attach your algorithm** — `doca_pcc_set_app(pcc, pcc_ecn_rp_app)`. That `pcc_ecn_rp_app` is the **compiled DPA image**, built from `device/rp_main.c` + your `device/algo/rtt_template.c` — this one line is where the code you write gets loaded in.
5. **Configure it** — the DPA thread pool, the CNP probe format (plain RoCE CNP), logging.
6. **Start it** — `doca_pcc_start()` uploads your algorithm onto the DPA and sets it running. **From this moment the DPA is in charge:** every congestion event runs your code.
7. **Supervise** — the host then just loops in `doca_pcc_wait()`, keeping the process healthy and otherwise doing nothing.

Step 6 is the handoff; everything you write runs on the DPA from there, once per event.

It runs once per event, per flow. The DPA calls your algorithm — the entry point `doca_pcc_dev_user_algo()` in `device/rp_main.c` — once per congestion-relevant event *for each flow*, not once per packet. It hands your code that flow's saved state, the event, and a `results` struct; you set the new rate by writing it into `results` (the function itself returns nothing). The events that matter here are:

- a **packet was sent** (a "TX" event),
- a **CNP arrived** — a Congestion Notification Packet, the receiver's way of saying "I got a CE-marked packet, you are causing congestion" (the mark you set in Part 1, echoed back).

  INFO — the NIC batches events for you. At line rate the DPA could never keep up with one event per packet, so the hardware **coalesces** them: one event stands for many packets. It reacts per *batch*, while the hardware rate limiter does the per-packet work.

How an event reaches your handler. You don't wire any of this up. The DPA calls one fixed entry point per event, and a small dispatch routes it to the right handler by event type:

Each of the two reactions below is just the body of one of those `..._handle_roce_*` functions in `device/algo/rtt_template.c` — the dispatch that gets you there is already written. **Both handlers**

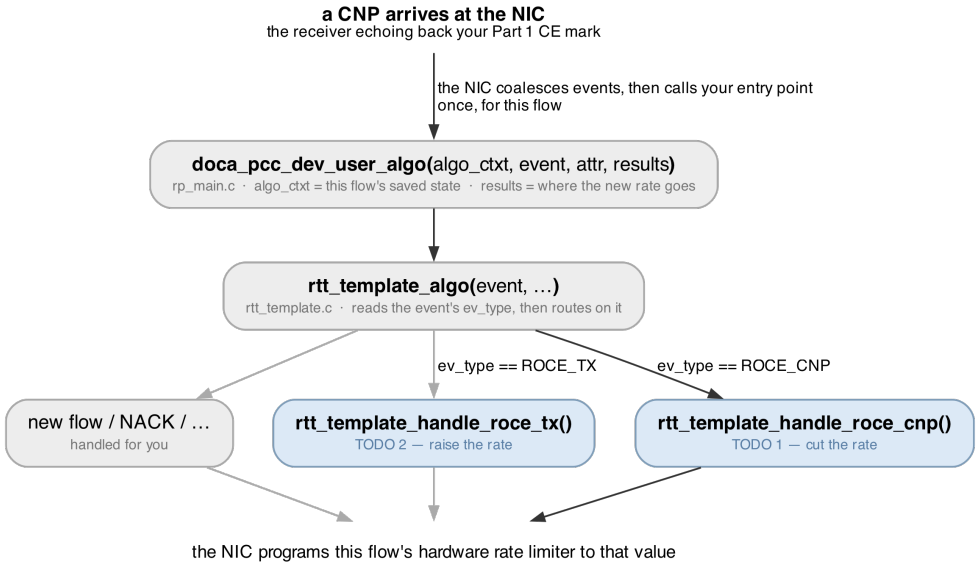

Figure 2: The path a CNP takes to the handler you write. The dark path is the one traced here; the pale branches are where the other events land. Whichever handler runs, it writes `results->rate`.

are left blank for you — that's the exercise. Here, just take in the shape.

1. A CNP arrives → cut the rate (multiplicative decrease). ← you write this, `TODO 1`. Congestion is happening, so the rate must come **down** by a fixed factor, and never fall below a floor. It goes in `..._handle_roce_cnp()`. The pieces, all already there for you:

- `cur_rate` — the flow's current rate (the fixed-point number from Step 1); you edit it in place.
- `ECN_CNP_DEC_FACTOR` — the cut factor, ×0.90 by default (a `#define` at the top of the file).
- `doca_pcc_dev_fxp_mult(factor, rate)` — multiplies that fixed-point `factor` into a rate for you.
- `MIN_RATE` — the floor the rate must not drop below.

2. Packets keep flowing, no congestion → raise the rate (additive increase). ← you write this, `TODO 2`. When traffic is moving and nothing is cutting it, the rate should **drift back up** — gently, and only occasionally, so it doesn't overshoot and immediately re-trigger congestion. It goes in `..._handle_roce_tx()`. The pieces:

- a `static` counter, so you act only every ~1000th call rather than on every single send event;
- `AI >> 2` — a small step to add each time, about 1.25% of line rate;
- `RATE_MAX` — the ceiling the rate must not exceed.

Put together, they make the DCQCN **sawtooth**: every CNP knocks the rate down by 10%, and between CNPs it drifts back up ~1.25% at a time. When the link is congested the down-cuts win and the rate settles low; when congestion clears (no more CNPs) the rate climbs back to full. The controller steers on the ECN signal alone (the CNPs) — which is what *pure-ECN* means.

> **INFO — the one knob that changes its personality.** The decrease factor is a single constant at the top of `rtt_template.c`:
>
> ```
> #define ECN_CNP_DEC_FACTOR (((1 << 16) * 900) / 1000)  // ×0.90 per CNP; 800..995
> ↪  = ×0.80..×0.995
> ```
>
> `900` means ×0.90. Make it smaller and each CNP cuts harder; larger and it barely reacts. You'll use this constant in **TODO 1**, and tune it in Step 4.3.

Step 4 — Write the two reactions

The controller is **almost complete** — the event loop, the logging, and the whole host loader are done. **The two reactions that actually move the rate are left for you**, both in `device/algo/rtt_template.c`: `TODO 1` (the cut) in `..._handle_roce_cnp()`, and `TODO 2` (the increase) in `..._handle_roce_tx()`. As shipped they're empty, so the controller **builds and runs but never changes the rate** — a flow sends flat-out no matter how congested the link is. You'll add them one at a time and watch each half of the loop come alive.

You'll have three things running together while you test:

what	where	why
the Part 1 marker (`doca_flow_ecn`)	PF0 (`mlx5_0`)	marks packets CE, so the receiver sends CNPs
the PCC controller (`doca_pcc_ecn_rp`)	PF1 (`mlx5_1`)	reacts to those CNPs by setting the rate
RoCE traffic (`ib_write_bw`)	the SFs (`mlx5_2`/`mlx5_3`)	the flow whose rate you'll watch move

> **NOTE — rebuild with `--reconfigure`.** Everything under `device/` — where both TODOs live — is compiled into the DPA image at **configure** time, not build time. Plain `ninja` has no dependency on those files, so on its own it will report `no work to do` and re-link the DPA image you built *before* your edit. After editing a TODO, always rebuild with:
>
> ```
> meson setup --reconfigure build && ninja -C build
> ```

Step 4.1 — `TODO 1`: cut the rate, and watch it collapse

In the Step 2 checkpoint you saw the scaffold run with its rate stay **flat** under a flood of CNPs — it sees congestion but never reacts. Now you write the reaction that makes it react.

Try it yourself! Make the controller react to congestion.

Get the flow running, and note the baseline. Bring the same setup back up as in the Step 2 checkpoint — the Part 1 marker at `--percent 100` and `./scripts/benchmark.sh` — and note the baseline throughput on its chart (~92 Gb/s, "full speed"). As you saw there, with `TODO 1` still empty the rate sits flat and the BW doesn't budge. Leave the traffic and marker running while you edit.

Write `TODO 1`. Open `device/algo/rtt_template.c` and find `TODO 1` in `..._handle_roce_cnp()`. Using the pieces from Step 3 — `doca_pcc_dev_fxp_mult()`, `ECN_CNP_DEC_FACTOR`, `cur_rate`, `MIN_RATE` — make the rate come **down** by the cut factor on each CNP, then clamp it up to the floor so it can't go below `MIN_RATE`. It's two lines. (Stuck? "Checking your work" below has it.)

Rebuild, reload, and watch it collapse (the traffic and marker from the previous step are still running):

```
meson setup --reconfigure build && ninja -C build
sudo stdbuf -oL ./build/doca-pcc-ecn/doca_pcc_ecn_rp -d mlx5_1 -l 50
```

Now you should see two things:

1. **The throughput drops sharply.** With the controller cutting the rate on every CNP, the client's BW average falls well below your baseline (it at least halves, usually much more). *That is congestion control happening* — the sender is throttling itself.
2. **The rate walking down** in the controller's terminal — the multiplicative-decrease half of the loop, live (the `PURE_ECN cnp=… rate=…` lines, no longer flat):
   `PURE_ECN cnp=1     rate=943718`

```
PURE_ECN cnp=501  rate=214380
PURE_ECN cnp=1001 rate=52428
```

Stop the controller with **Ctrl-C** (SIGINT — the graceful stop) when you're done looking.

Step 4.2 — `TODO 2`: raise the rate, and complete the sawtooth

With only `TODO 1`, the rate can only ever fall: once CNPs cut it, it stays down even after congestion clears. `TODO 2` adds the other half — the gentle climb back up — so the controller can *recover*.

Try it yourself! Let the rate come back.

Write `TODO 2`. Find `TODO 2` in `..._handle_roce_tx()`. Using the pieces from Step 3 — a `static` counter, `AI >> 2`, `RATE_MAX` — every ~1000th call, add the small step to `cur_rate` and cap it at `RATE_MAX`. Acting only every ~1000th call is what keeps the increase *gentle*; leave the gate out and the rate rockets straight back up and re-triggers congestion.

Rebuild: `meson setup --reconfigure build && ninja -C build`.

Run, and this time make congestion come and go. Load the controller (foreground) and traffic as in Step 4.1, then, partway through, **stop the marker** so the CNPs dry up while the flow keeps running:

```
sudo pkill -INT -x doca_flow_ecn    # congestion clears; no more CNPs
```

The climb back shows up on `benchmark.sh`'s chart, **not** in the controller's trace — `PURE_ECN` only prints when a CNP arrives, so the moment the CNPs stop the trace stops with them. Watch the throughput instead: it collapses under marking (`TODO 1`), then climbs back toward line rate once the marker is gone (`TODO 2`). That fall and rise is the DCQCN **sawtooth** — both halves of your controller working together:

```
PURE_ECN cnp=1001 rate=52428      # cut down while marking
# ... marker stopped: no new PURE_ECN cuts, TX events raise rate to 1048576 ...
```

Stop with **Ctrl-C** (SIGINT — the graceful stop).

Step 4.3 — Tune how hard it cuts (optional)

Now that the loop works, change **how aggressively** it reacts. Every CNP currently cuts the rate by 10% (`ECN_CNP_DEC_FACTOR` = ×0.90). This one number sets the controller's whole personality: a sharper cut empties the queue faster (fewer retransmissions, higher goodput, lower latency) up to a point; too sharp and you under-use the link.

Try it yourself! Sweep the cut factor.

Open `device/algo/rtt_template.c` and change the `900` in `ECN_CNP_DEC_FACTOR` (near the top of the file). Try a gentler reaction first, rebuild, and re-run Step 4.1:

```
#define ECN_CNP_DEC_FACTOR (((1 << 16) * 990) / 1000)  // ×0.99 — barely cuts per CNP
```

With ×0.99 the rate barely comes down, the queue stays deep, and you'll see **far more CNPs** in the controller's `PURE_ECN` trace and lower goodput than at ×0.90. Now try a **sharper** cut (`800` = ×0.80) and compare — fewer CNPs, a shallower queue. The tuned sweet spot on our testbed was **×0.90**:

per-CNP cut	goodput	CNPs	latency
×0.99	68 Gb/s	many	high
×0.90	**88.5 Gb/s**	few	**low**
×0.80	86.7 Gb/s	fewest	very low

> **INFO — the surprising bit.** The rate *set-point* averaged about the same across all of these, yet goodput ranged from 68 to 88 Gb/s. Goodput is governed by **queue depth and retrans-**

missions, not the average rate — which is why a *sharper* cut (shallower queue, fewer drops) can *raise* goodput. We have a script that sweeps this automatically and plots the whole curve — ask an organiser if you would like to see it.

Put `900` back when you're done to restore the tuned controller.

Checking your work

The program tells you where you stand at every step: the throughput dropping ends Step 4.1, the throughput recovering when the marker stops ends Step 4.2. When one of those doesn't happen, the Debugging tips below name the usual cause.

The two finished reactions, if you want to compare:

`TODO 1` — in `..._handle_roce_cnp()`:

```
cur_rate = doca_pcc_dev_fxp_mult(ECN_CNP_DEC_FACTOR, cur_rate);   // rate = rate × 0.90
if (cur_rate < MIN_RATE) cur_rate = MIN_RATE;                     // never below the floor
```

`TODO 2` — in `..._handle_roce_tx()`:

```
static uint32_t g_tx_inc = 0;
if ((++g_tx_inc % 1000) == 0) {          // every ~1000th send event
    cur_rate += (AI >> 2);               // add ~1.25% of line rate
    if (cur_rate > RATE_MAX) cur_rate = RATE_MAX;
}
```

Or diff your file against the finished controller — it ships in the **solutions** repository at the same path:

```
diff device/algo/rtt_template.c \
  <solutions-repo>/doca-2/doca-pcc-ecn/device/algo/rtt_template.c
```

Ask an organiser if you're stuck. That's what we're here for.

Debugging tips

- **The rate never moves — `PURE_ECN` prints the same number on every line** → `TODO 1` is still empty (or you edited it but didn't rebuild). That flat output is the scaffold's as-shipped behaviour; write the multiplicative decrease in `..._handle_roce_cnp()` and rebuild. If instead the rate falls but never climbs back after congestion clears, `TODO 2` (the increase, in `..._handle_roce_tx()`) is the missing half.
- **An edit didn't take effect and `ninja` said `no work to do`** → you rebuilt with plain `ninja`. The DPA image is built at *configure* time, and nothing under `device/` is in ninja's dependency graph, so it cannot see your edit. Rebuild with `meson setup --reconfigure build && ninja -C build`. (`rm -rf build && meson setup build` also works and is what to reach for if the build itself looks confused, but it is far slower.)
- **The controller exits a second or two after starting** → the firmware knob `USER_PROGRAMMABLE_CC` is not live. Check with `admin/local_scripts/check_pcc_ready.sh` (it should say `ready`). On the tutorial cards it's set for you.
- **No `PURE_ECN` lines ever appear** → no CNPs are reaching the controller. Make sure the Part 1 marker (`doca_flow_ecn --percent 100`) is running so packets are CE-marked, and that traffic is actually flowing. (If the output is just too chatty, reload with `| grep --line-buffered PURE_ECN` appended to see only that trace.)
- **Traffic must use `-R`** (`benchmark.sh` and `run_{server,client}.sh` already do). Without it the flow isn't bound to your algorithm and your handlers never run — the rate won't move at all.
- **Stop it gracefully with Ctrl-C** in its terminal (SIGINT), or with `sudo pkill -INT -x`

`doca_pcc_ecn_rp` from another terminal. A hard `kill -9` (or `docker rm -f`) leaves a "ghost" program loaded on the DPA that blocks the next run until a chip reset.
- **Run it in the foreground** (as shown), so its output prints live. Launched in the background over SSH it can appear to die after a few seconds — that's the launch, not the program.

What you built

- The **two-halves** model: a host loader ships your algorithm onto the **DPA**, which runs it once per event and sets a per-flow **rate** that the NIC hardware then enforces.
- The two reactions **you wrote** — `TODO 1` (CNP → cut ×0.90) and `TODO 2` (TX → raise ~1.25%) — which together make a complete **DCQCN** sawtooth.
- **Congestion control happening live**: with only the cut written, a flow's throughput collapsing as the controller reacts to the CE marks you produced in Part 1; with the recovery added too, the rate climbing back when congestion clears.
- How **one constant** changes the whole behaviour, and why a sharper cut can *improve* goodput.

In **Part 1** you marked the congestion signal in the data plane, and here in **Part 2** you wrote and tuned the controller that reacts to it on the DPA — the two halves of programmable transport on a SmartNIC.

Appendix A — The DPA and the two halves, in more detail

Reference material. You do not need any of this to finish the exercise.

Why two halves, and why the DPA. The Arm cores are general-purpose Linux CPUs, too far from the wire to make a per-flow rate decision fast enough. The **DPA** (Data-Path Accelerator) is a cluster of many-threaded RISC-V processors sitting on the data path, built for tiny, frequent, reactive computation. So the algorithm runs there, and the host program on the Arm is only a loader and supervisor: it compiles the DPA image (`dpacc`), uploads it, opens the PCC context, and then keeps the process alive. Nothing on the host runs per event.

`USER_PROGRAMMABLE_CC`. Running your *own* congestion-control code on the DPA is gated by a NIC firmware setting, `USER_PROGRAMMABLE_CC=1`. Without it, `doca_pcc_ecn_rp` refuses to start (it exits a second or two in). It only takes effect after a full power-cycle of the card, so it's set up for you ahead of the tutorial. Check it with `admin/local_scripts/check_pcc_ready.sh` — it should say `ready`.

Why the controller attaches to PF1 but traffic uses the SFs. Congestion control is a property of the *port/uplink*, so the controller attaches to the sender's physical function, PF1 (`mlx5_1`). The traffic rides on sub-functions of that port (`mlx5_2`/`mlx5_3`). One controller governs all the flows on its port.

Appendix B — The controller's model in full

Reference material. Step 3 has the working subset.

Rate is a fixed-point fraction. The rate is not bits per second; it is a fixed-point number where `1 << 20` (1,048,576) is 100% of line rate. `524288` is 50%, a small floor value is a near-stop. Your algorithm writes this number into `results->rate`; the NIC's hardware rate limiter enforces it until your code runs again.

Events are coalesced. Your algorithm does **not** run per packet — at line rate the DPA could never keep up. The hardware batches events so one event stands for many packets, and your handler runs per *batch* while the rate limiter does the per-packet pacing. That two-tier split is exactly why it keeps up at line rate.

Why the rate collapses so hard at `--percent 100`. Marking *every* packet makes the receiver send a constant stream of CNPs, so the controller thinks the link is maximally congested and keeps cutting. A smaller fraction (`--percent 50`) is a gentler, more realistic signal.

Rate set-point vs measured goodput. The rate is a *set-point*; the throughput you actually get also depends on queueing and retransmissions. A controller can hold a similar average rate yet deliver very different goodput — which is what the Step 4.3 sweep shows: a sharper cut keeps the queue shallow and can *raise* goodput even though the average rate is unchanged.

Appendix C — The code you edit, and the API

The DPA headers on the card are the authority. This is only a map of what matters here.

The file you edit is `device/algo/rtt_template.c`. Its two handlers are the whole exercise:

Function	Event	You write
`..._handle_roce_cnp()`	a CNP	`TODO 1` — multiplicative decrease (cut ×0.90)
`..._handle_roce_tx()`	a send	`TODO 2` — additive increase (raise ~1.25%)

The dispatch that routes an event to the right handler, and the entry point `doca_pcc_dev_user_algo()` (in `device/rp_main.c`), are already written for you.

The DPA helpers and constants you use:

Name	What it is
`cur_rate`	the flow's current rate (fixed-point); edit it in place
`doca_pcc_dev_fxp_mult(f,r)`	multiply a fixed-point factor `f` into a rate `r`
`ECN_CNP_DEC_FACTOR`	the per-CNP cut factor (×0.90 by default); the one tuning knob
`MIN_RATE` / `RATE_MAX`	the floor and ceiling the rate must stay within
`AI`	the additive-increase step; `AI >> 2` is ~1.25% of line rate

The host loader (`host/pcc_ecn_rp.c`, which you don't edit) uses `doca_pcc_create`, `doca_pcc_set_app`, `doca_pcc_start`, and `doca_pcc_wait` — open the context, attach the DPA image, start it, supervise.

The program's own flags: `-d <device>` (which NIC, e.g. `mlx5_1`) and `-l <level>` (log verbosity).

Further reading

- DOCA PCC programming guide — search "DOCA PCC"; swap the archive version to match your card.
- DOCA DPA subsystem — how DPA programs are built and loaded.